

VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Dining Philosophers Simulator

A9510 – Operating Systems Laboratory

Submitted by

Batch No: 07

Ayush Reddy AK 25881A0585

G. Karthik 25881A0586

Course Facilitator

Ms. Chitrala Kavitha
Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

VARDHAMAN COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A++** Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

Vibrant Vardhaman

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Course End Project titled **"Dining Philosophers Simulator"** is carried out by **Ayush Reddy AK and G. Karthik**, with Roll Numbers **25881A0585 and 25881A0586**, towards **A9510 – Operating Systems Laboratory** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic Year 2026–27.

Course Facilitator	Academic Head	Head of the Department
Ms. Chitrala Kavitha	**Dr. R. Karthikeyan**	**Prof. Y. Vijayalata**
Assistant Professor, CSE	Professor, CSE	HoD, CSE

Contents

Abstract . . . 1
1 Introduction . . . 2
1.1 Background . . . 2
1.2 The Classical Problem . . . 2
1.3 Problem Statement . . . 3
1.4 Motivation and Relevance . . . 3
1.5 Scope of the Project . . . 3
1.6 Report Organisation . . . 3
2 Objectives and Requirements . . . 4
2.1 Primary Objectives . . . 4
2.2 Functional Requirements . . . 4
2.3 Non-Functional Requirements . . . 4
2.4 Development Environment . . . 5
3 Theoretical Background and Literature Review . . . 5
3.1 Concurrency and Parallelism . . . 5
3.2 Threads and Shared Address Space . . . 5
3.3 Critical Sections and Mutual Exclusion . . . 6
3.4 POSIX Threads . . . 6
3.5 Deadlock . . . 6
3.6 Starvation and Fairness . . . 7
3.7 Resource Allocation Graph . . . 7
3.8 Comparison of Common Solutions . . . 7
3.9 Summary of the Review . . . 8
4 System Architecture and Design . . . 8
4.1 High-Level Architecture . . . 8
4.2 Core Entities . . . 9
4.2.1 Main Thread . . . 9
4.2.2 Philosopher Threads . . . 9
4.2.3 Fork Mutexes . . . 9
4.2.4 Console Output . . . 9
4.3 Data Structures . . . 9
4.4 State Model . . . 10
4.5 Design Invariants . . . 10
5 Methodology and Algorithm . . . 11
5.1 Overall Workflow . . . 11
5.2 Fork Index Calculation . . . 11

5.3 Asymmetric Acquisition Algorithm . . . 12
5.4 Why the Strategy Prevents Deadlock . . . 12
5.5 Thinking and Eating Delays . . . 12
5.6 Thread Creation and Joining . . . 13
5.7 Complexity Analysis . . . 13
6 Implementation . . . 13
6.1 Implementation Overview . . . 13
6.2 Explanation of the Worker Function . . . 14
6.3 Important Implementation Considerations . . . 14
6.3.1 Shared Output . . . 14
6.3.2 Random Number Generation . . . 15
6.3.3 Error Handling . . . 15
6.3.4 Correct Identifier Passing . . . 15
6.4 Compilation and Execution . . . 15
6.5 Suggested Complete-Program Structure . . . 15
7 Testing and Results . . . 16
7.1 Testing Objectives . . . 16
7.2 Build Test . . . 16
7.3 Sample Simulation Output . . . 16
7.4 Test Cases . . . 17
7.5 Results . . . 17
7.6 Interpretation of Results . . . 18
7.7 Limitations of the Evaluation . . . 18
8 Discussion . . . 18
8.1 Mutual Exclusion in Practice . . . 18
8.2 Deadlock Prevention Versus Deadlock Detection . . . 18
8.3 Starvation and the Limits of Random Delays . . . 19
8.4 Scheduling and Nondeterminism . . . 19
8.5 Performance Considerations . . . 19
8.6 Reliability and Defensive Programming . . . 19
9 Advantages, Limitations, and Applications . . . 20
9.1 Advantages . . . 20
9.2 Limitations . . . 20
9.3 Real-World Applications . . . 20
9.4 Learning Outcomes . . . 21
10 Future Enhancements . . . 21
10.1 Configurable Number of Philosophers . . . 21
10.2 Graphical Visualisation . . . 21
10.3 Waiting-Time Statistics . . . 21
10.4 Fair Waiter Algorithm . . . 21
10.5 Alternative Synchronization Strategies . . . 22
10.6 Improved Error Handling and Instrumentation . . . 22
11 Conclusion . . . 22
Bibliography . . . 23

List of Figures

1 Conceptual arrangement of five philosophers and five shared forks. 2
2 High-level architecture of the Dining Philosophers simulator. 9
3 Conceptual state transitions of a philosopher. 10

List of Tables

1 Functional requirements of the simulator. 4
2 Comparison of common Dining Philosophers strategies. 8
3 Proposed functional and behavioural test cases. 17

Abstract

The Dining Philosophers problem is a classic synchronization problem in operating systems that illustrates the challenges involved when multiple concurrent processes compete for a finite set of shared resources. The problem is useful for understanding mutual exclusion, deadlock, starvation, resource allocation, and the interaction between application threads and the operating system scheduler. This project develops a C-based simulation in which five philosophers are represented by POSIX threads and five forks are represented by mutex locks.

Each philosopher alternates between thinking and eating. A philosopher may enter the eating state only after acquiring both adjacent forks. Since each fork is shared by two neighbouring philosophers, the program must coordinate access to these resources without allowing two threads to use the same fork simultaneously. The implementation uses `pthread_create`, `pthread_join`, and `pthread_mutex_lock`/`pthread_mutex_unlock` to create threads and manage shared resources. It adopts an asymmetric fork-acquisition rule: even-numbered philosophers acquire the right fork before the left, while odd-numbered philosophers acquire the left fork before the right. This breaks the circular-wait condition required for deadlock and therefore prevents deadlock under the stated locking design.

Randomized delays are used to make the simulation's interleaving more visible and less predictable. These delays model time spent thinking and eating; however, random delays alone do not provide a formal guarantee against starvation. The report explains this distinction, describes the architecture and algorithm, discusses the implementation, and provides a testing plan and analysis framework. The project demonstrates how synchronization primitives can coordinate concurrent execution and why a correct resource-acquisition policy is essential in multithreaded software.

Keywords: Operating Systems, Dining Philosophers, POSIX Threads, Mutex, Synchronization, Deadlock Prevention, Starvation, Concurrency, Resource Allocation.

1. Introduction

1.1 Background

Modern operating systems allow several processes and threads to make progress during overlapping periods of time. On a multicore system, different threads may execute simultaneously; on a single-core system, the scheduler can switch rapidly between runnable threads to create the appearance of concurrent progress. Concurrency improves responsiveness and can improve throughput, but it introduces coordination challenges whenever threads access shared state or compete for limited resources.

Examples of shared resources include files, memory structures, database records, printers, network connections, and hardware devices. If multiple threads modify the same data without appropriate coordination, the result may depend on the order in which their instructions happen to execute. Such timing-dependent errors are commonly described as race conditions. Locks and other synchronization primitives help developers define which operations may occur at the same time and which operations must be performed exclusively.

The Dining Philosophers problem is a compact model of these issues. Although the setting is a dining table, the central problem is not about food. It is about a set of independent agents that each require more than one resource to complete an operation. Similar situations occur when a program must lock two data structures, when a transaction needs several records, or when a service requires multiple exclusive devices before proceeding.

1.2 The Classical Problem

Five philosophers sit around a circular table. One fork is placed between every pair of neighbouring philosophers, giving five philosophers and five forks in total. A philosopher repeatedly thinks and eats. Thinking does not require a fork, while eating requires both the fork on the left and the fork on the right.

Each fork can be held by at most one philosopher at a time. Therefore, the program must enforce mutual exclusion for each fork. A philosopher who has acquired one fork but cannot acquire the other must wait. If all philosophers acquire one fork and then wait for the other, no philosopher can proceed; the system is deadlocked.

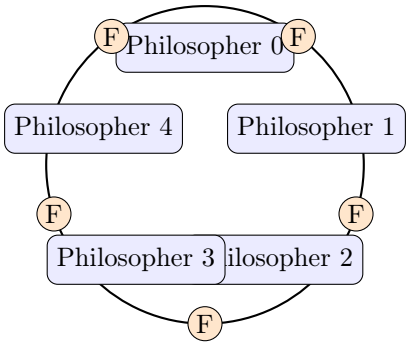


Figure 1: Conceptual arrangement of five philosophers and five shared forks.

1.3 Problem Statement

The objective is to implement a concurrent simulation that allows five philosopher threads to think and eat while sharing five forks. The implementation must prevent simultaneous use of the same fork and avoid deadlock. It should also provide observable output that helps a reader understand the sequence of state changes and resource acquisition.

The project uses a mutex for every fork and an asymmetric acquisition order to prevent circular wait. The implementation is finite: each philosopher performs a fixed number of cycles and then terminates. This makes the program convenient for laboratory demonstration and allows the main thread to join all worker threads before cleaning up the mutexes.

1.4 Motivation and Relevance

The problem is often introduced in operating systems courses because it combines several important concepts in a small example. A solution must do more than prevent two threads from using the same fork. It must also consider the order in which resources are requested, the possibility of indefinite waiting, and the lifecycle of concurrent threads.

The same reasoning is useful when designing systems that use multiple locks. If different threads acquire the same locks in inconsistent orders, a circular dependency may form. Establishing a consistent or deliberately asymmetric ordering can eliminate such cycles. Studying the Dining Philosophers problem therefore helps build the habit of analysing resource dependencies before writing concurrent code.

1.5 Scope of the Project

The project focuses on a terminal-based C program running on a POSIX-compatible operating system. The simulation uses five philosopher threads, five mutexes, randomized delays, and a fixed number of eating cycles. It does not attempt to model the full complexity of a real dining environment or a production scheduler.

The program is designed as an educational demonstration rather than a real-time fairness system. In particular, the asymmetric locking rule prevents deadlock, but it does not by itself guarantee that every philosopher receives a fork within a fixed time. A stronger fairness policy would require an additional mechanism such as a waiter, a semaphore-based admission controller, or a queue that grants access in order.

1.6 Report Organisation

Section 2 defines the objectives and requirements. Section 3 reviews the concepts needed to understand the problem. Section 4 describes the system architecture and design. Section 5 explains the methodology and algorithms. Section 6 presents the implementation. Section 7 describes the testing approach and sample output. Section 8 discusses observations, limitations, and possible improvements. The final sections present the conclusion, future scope, and bibliography.

2. Objectives and Requirements

2.1 Primary Objectives

The main objectives of the project are:

1. To model the classical Dining Philosophers problem using the C programming language.
2. To represent each philosopher as an independent POSIX thread.
3. To represent each fork as a mutex and prevent simultaneous access to the same fork.
4. To implement asymmetric fork acquisition so that the circular-wait condition is broken.
5. To model thinking and eating periods using delays and display state transitions in the terminal.
6. To wait for all worker threads to finish and release synchronization resources safely.
7. To analyse the difference between deadlock prevention and starvation prevention.

2.2 Functional Requirements

The functional requirements describe what the program must do.

ID	Requirement
FR-01	The program shall initialise five philosopher records and five fork mutexes.
FR-02	The program shall create one thread for each philosopher.
FR-03	Each philosopher shall display a thinking message before attempting to eat.
FR-04	Each philosopher shall acquire both adjacent forks before entering the eating state.
FR-05	A fork shall not be held by more than one philosopher at a time.
FR-06	The fork-acquisition order shall depend on whether the philosopher identifier is even or odd.
FR-07	Each philosopher shall release both forks after eating.
FR-08	Each philosopher shall repeat the configured number of cycles and then terminate.
FR-09	The main thread shall join all successfully created philosopher threads.
FR-10	The program shall destroy the mutexes after the worker threads have completed.

Table 1: Functional requirements of the simulator.

2.3 Non-Functional Requirements

- **Correctness:** Each philosopher must hold both adjacent forks before eating.

- **Mutual exclusion:** The same fork must never be allocated to two philosophers simultaneously.
- **Deadlock avoidance:** The chosen acquisition policy must prevent a circular wait among philosophers.
- **Readability:** The source code and terminal messages should be clear enough for laboratory evaluation.
- **Portability:** The program should compile on a POSIX-compatible system with a C compiler and pthread support.
- **Resource cleanup:** Threads and mutexes should be handled through an orderly lifecycle.

2.4 Development Environment

The program is intended to run on Linux or another POSIX-compatible operating system. GCC is a suitable compiler, and the pthread library provides the thread and mutex APIs. A typical compilation command is:

```
gcc dining_philosophers.c -o dining -pthread
```

Listing 1: Compilation command

The `-pthread` option informs the compiler and linker that POSIX thread support is required. The output executable can then be launched from a terminal. The precise compiler version and operating-system release should be recorded by the project team if reproducibility is required.

3. Theoretical Background and Literature Review

3.1 Concurrency and Parallelism

Concurrency refers to multiple tasks making progress during overlapping periods. Parallelism refers to tasks executing at the same instant on separate processing resources. A multithreaded program may be concurrent even when it runs on a single CPU core, because the scheduler switches between runnable threads. On a multicore processor, some threads may also run in parallel.

The distinction matters because correctness cannot rely on a particular execution order. A thread may be paused between any two operations, and another thread may run before the first resumes. A correct synchronization design must remain valid across many possible interleavings.

3.2 Threads and Shared Address Space

A thread is an execution path within a process. Threads in the same process typically share the process's address space, including global variables and dynamically allocated memory, while maintaining separate stacks and execution contexts. This shared memory makes communication efficient, but it also means that shared objects must be protected when concurrent access could conflict.

In this project, all philosopher threads access a shared array of mutexes. Each thread also receives an identifier so that it can determine which forks it needs. The array is shared, but the mutex implementation coordinates access to each individual fork.

3.3 Critical Sections and Mutual Exclusion

A critical section is a region of code that accesses a shared resource and must obey a synchronization rule. In the Dining Philosophers simulator, the logical critical section is the eating period, because a philosopher must own both adjacent forks while eating. A mutex provides a lock that one thread can hold at a time. A thread attempting to lock an already-locked mutex waits until the lock becomes available.

Mutual exclusion is a safety property: it prevents incompatible operations from happening simultaneously. For the simulator, the key invariant is that a fork has at most one owner at any time. The program must lock both forks before eating and unlock them afterward, including in any expanded implementation that handles errors.

3.4 POSIX Threads

POSIX threads, commonly called pthreads, are a standard interface for creating and coordinating threads on POSIX systems. The main operations used in this project are:

- `pthread_create`: starts a new thread that executes a specified function.
- `pthread_join`: waits for a thread to terminate and collects its completion status where applicable.
- `pthread_mutex_init`: initialises a mutex object.
- `pthread_mutex_lock`: acquires a mutex or waits until it can be acquired.
- `pthread_mutex_unlock`: releases a mutex held by the calling thread.
- `pthread_mutex_destroy`: destroys a mutex after it is no longer in use.

These functions return status codes. Robust production code should check the return value of every operation and handle partial initialisation or thread-creation failure. The compact laboratory version may omit some of this defensive code for clarity, but the limitation should be acknowledged.

3.5 Deadlock

A deadlock is a state in which a set of threads is permanently blocked because each thread is waiting for a resource held by another thread in the set. The classical Coffman conditions describe four conditions that must hold simultaneously for deadlock to be possible:

1. **Mutual exclusion:** At least one resource is non-shareable.
2. **Hold and wait:** A thread holds one or more resources while waiting for additional resources.
3. **No preemption:** Resources cannot be forcibly taken away; they are released voluntarily.

4. **Circular wait:** A cycle exists in which each thread waits for a resource held by the next thread.

A deadlock-prevention strategy ensures that at least one of these conditions cannot hold. This project retains mutual exclusion, hold and wait, and no preemption, but breaks circular wait through asymmetric ordering.

3.6 Starvation and Fairness

Starvation occurs when a thread waits indefinitely or for an unacceptably long period because other threads repeatedly obtain the resource first. Deadlock and starvation are different. A deadlocked system has a group of threads that cannot progress, whereas a system affected by starvation may continue making progress overall while one particular thread is continually denied access.

Random sleep intervals can vary execution timing and make the output more representative of concurrent activity, but they do not guarantee fairness. POSIX mutexes do not universally promise strict FIFO acquisition. A thread can therefore theoretically be delayed repeatedly even when the system as a whole continues to make progress. A formal starvation-freedom guarantee would require a policy that explicitly manages waiting order or resource admission.

3.7 Resource Allocation Graph

A resource-allocation graph is a directed graph used to represent the relationship between processes or threads and resources. A request edge indicates that a thread is waiting for a resource, while an assignment edge indicates that a resource is allocated to a thread. In a single-instance resource system, a cycle in the graph is a useful indication of possible deadlock. In the Dining Philosophers problem, each fork is a single-instance resource and each philosopher may request two forks.

The asymmetric policy imposes a direction on the order of requests. Because neighbouring philosophers do not all request forks in the same direction, the system cannot form the same circular chain in which every philosopher holds one fork and waits for the next. This illustrates how resource-ordering rules can simplify deadlock prevention.

3.8 Comparison of Common Solutions

Several strategies can be used to solve the Dining Philosophers problem. Each has different trade-offs.

Approach	**Main idea**	**Strength**	**Limitation**
Naive locking	Every philosopher picks up the same side first.	Simple to implement.	Can create circular wait and deadlock.
Asymmetric ordering	Some philosophers reverse their fork order.	Small change; breaks circular wait in this arrangement.	Does not ensure strict fairness.
Global resource ordering	Every fork has an order; threads lock lower-numbered forks first.	General technique that prevents lock-order cycles.	Requires consistent ordering across all code paths.

Waiter or arbitrator	A central controller grants permission to attempt acquisition.	Can control admission and reduce contention.	Adds a central coordination point.
Semaphore limit	Permit only a bounded number of philosophers to compete for forks.	Simple way to reduce the circular-wait scenario.	Fairness depends on semaphore semantics and design.
Monitor solution	A monitor tracks philosopher states and grants access when neighbours are not eating.	Centralises state and conditions.	Requires condition-variable coordination.

Table 2: Comparison of common Dining Philosophers strategies.

3.9 Summary of the Review

The classical problem demonstrates that thread safety depends on both the synchronisation primitive and the protocol surrounding it. Mutexes enforce exclusive access to individual forks, while the asymmetric acquisition rule addresses the larger dependency structure. The distinction between safety, liveness, and fairness is important: mutual exclusion is a safety property, deadlock prevention supports system progress, and starvation freedom requires a stronger fairness argument.

4. System Architecture and Design

4.1 High-Level Architecture

The simulator is divided conceptually into an initialisation stage, a thread-execution stage, and a cleanup stage. The main thread prepares shared resources and starts the philosopher threads. Each philosopher independently runs the same thread function with a different identifier. Mutexes coordinate access to forks, and terminal messages expose the state transitions to the observer.

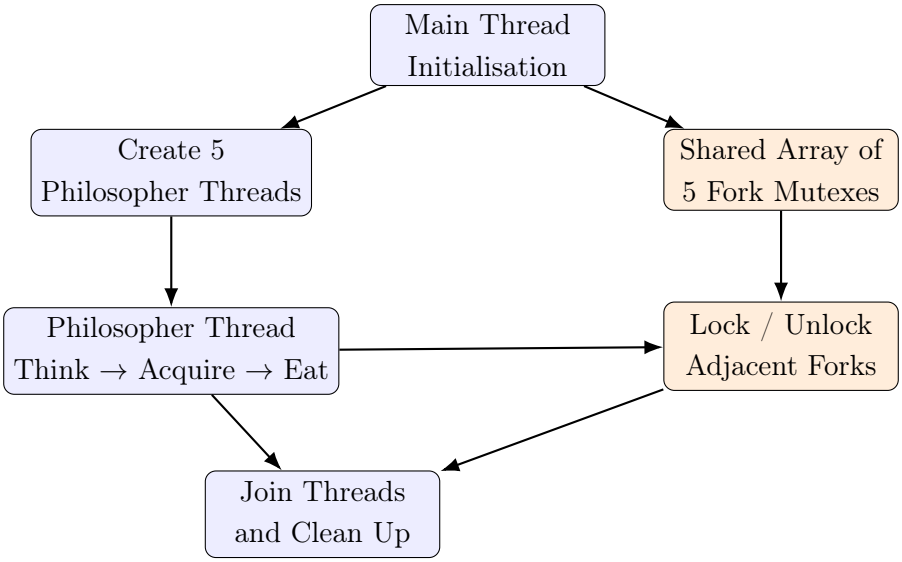


Figure 2: High-level architecture of the Dining Philosophers simulator.

4.2 Core Entities

4.2.1 Main Thread

The main thread is responsible for setting up the simulation. It initialises the fork mutexes, prepares philosopher identifiers, starts worker threads, waits for their completion, and destroys the mutexes. It does not perform the philosophers' eating cycles itself.

4.2.2 Philosopher Threads

Each philosopher is represented by one thread. The thread function receives a pointer to the philosopher's identifier and calculates the indices of the two adjacent forks. It then repeats the think-and-eat cycle a fixed number of times. Since all threads execute the same function, the implementation is easier to maintain and compare.

4.2.3 Fork Mutexes

The fork array contains one mutex per fork. For philosopher i, the left fork index is i, while the right fork index is $(i+1) \bmod N$, where N is the number of philosophers. The modulo operation connects the last philosopher to the first fork, preserving the circular table arrangement.

4.2.4 Console Output

The console is the user interface for the simulation. Messages indicate when a philosopher begins thinking, begins eating, finishes eating, and exits. The output is not a deterministic trace: the order of messages can vary between executions because the operating system schedules threads and the delay durations may differ.

4.3 Data Structures

The essential data structures are a fixed-size mutex array, a thread identifier array, and an integer array containing philosopher IDs. The number of philosophers should be defined once

as a constant so that the code does not use inconsistent values in different locations.

```c
#define NUM_PHILOSOPHERS 5
#define EAT_CYCLES 3

pthread_mutex_t forks[NUM_PHILOSOPHERS];
pthread_t philosophers[NUM_PHILOSOPHERS];
int philosopher_ids[NUM_PHILOSOPHERS];
```

Listing 2: Illustrative declarations for the shared state

A fixed-size array is appropriate for a small, predefined simulation. A larger implementation could allocate these structures dynamically based on a user-specified number of philosophers.

4.4 State Model

Each philosopher moves through a repeating set of conceptual states:

- **Thinking:** The philosopher is not holding either fork and is not requesting resources.
- **Hungry / waiting:** The philosopher attempts to acquire the two required forks. This state may be brief or may last while another thread holds a required mutex.
- **Eating:** The philosopher owns both adjacent forks and performs the simulated eating delay.
- **Finished:** The philosopher has completed all configured cycles and returns from the thread function.

The program may not explicitly store these states in an enum or shared array; they are represented by control flow and output messages. An expanded graphical simulator could store each state explicitly and display it visually.

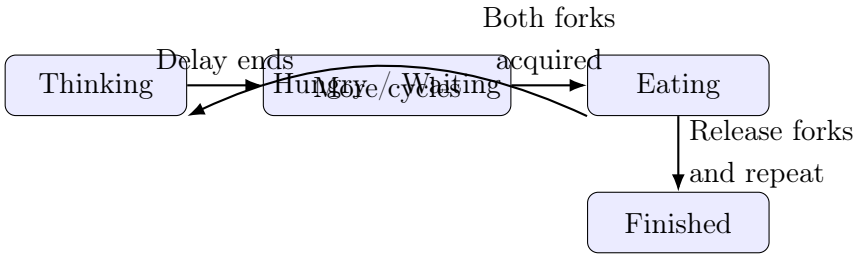


Figure 3: Conceptual state transitions of a philosopher.

4.5 Design Invariants

The implementation is based on the following invariants:

1. A philosopher must not enter the eating section until both adjacent fork mutexes have been acquired.
2. A mutex must be unlocked only by the thread that successfully acquired it.
3. A philosopher releases both forks after the eating interval.
4. The main thread must not destroy mutexes while philosopher threads may still use them.

5. Each thread uses a valid, stable identifier for the duration of its execution.

These invariants are useful when reviewing the code and when planning tests. They also help identify why thread joining is an essential part of resource cleanup.

5. Methodology and Algorithm

5.1 Overall Workflow

The methodology follows a sequence of setup, concurrent execution, and teardown. First, the program initialises all fork mutexes. Next, it creates the five philosopher threads. Each thread enters its loop, waits for a random interval while thinking, acquires its forks according to the asymmetric rule, simulates eating, and releases both forks. Once all cycles are complete, each thread returns. The main thread joins every worker and then destroys the mutexes.

1. Define the number of philosophers and the number of cycles.
2. Initialise one mutex for each fork.
3. Assign a unique identifier to each philosopher.
4. Create one thread for each philosopher.
5. In each thread, calculate the left and right fork indices.
6. Simulate thinking using a delay.
7. Acquire the two forks in the order selected by the philosopher's parity.
8. Simulate eating while holding both forks.
9. Release both forks and repeat until the cycle count is reached.
10. Join all created threads and destroy the mutexes.

5.2 Fork Index Calculation

For N philosophers, philosopher i uses:

$$\text{left}(i) = i$$

$$\text{right}(i) = (i + 1) \bmod N$$

For five philosophers, philosopher 0 uses forks 0 and 1, philosopher 1 uses forks 1 and 2, and so on. Philosopher 4 uses forks 4 and 0. The modulo operation is important because it closes the circle at the final philosopher.

5.3 Asymmetric Acquisition Algorithm

A naive solution might instruct every philosopher to pick up the left fork first. Under a particular timing, every philosopher could acquire their left fork before attempting the right fork. Each would then wait for a fork held by a neighbour, creating circular wait.

The implementation breaks this symmetry. If the philosopher ID is even, it locks the right fork first and the left fork second. If the ID is odd, it locks the left fork first and the right fork second. This means that the dependency direction is not identical for all philosophers.

```
if philosopher_id is even:
    lock(right_fork)
    lock(left_fork)
else:
    lock(left_fork)
    lock(right_fork)

eat()

unlock(left_fork)
unlock(right_fork)
```

Listing 3: Pseudocode for asymmetric fork acquisition

The unlock order does not need to be the reverse of the lock order for ordinary mutexes, provided the current thread owns both locks. Releasing both forks promptly is the key requirement. In a more complex program, cleanup should also handle failures between acquiring the first and second mutex so that a thread does not accidentally retain a fork.

5.4 Why the Strategy Prevents Deadlock

Deadlock requires circular wait. If every philosopher followed the same fork order, it would be possible for each to hold one fork and wait for the next. The asymmetric rule breaks that uniform cycle. At least one neighbouring relationship has a different acquisition direction, so the circular chain of dependencies cannot be maintained in the same way.

This is a prevention argument for the defined five-philosopher arrangement and locking policy. It is not a general proof that any arbitrary asymmetric rule prevents deadlock in every program. For more complicated systems, a global lock hierarchy is often easier to reason about: every thread must acquire resources in a consistent order based on a unique resource rank.

5.5 Thinking and Eating Delays

The simulation uses random delays such as `sleep(rand() % 2 + 1)` to pause a thread for one or two seconds. Thinking delays occur before resource acquisition, while eating delays occur after both forks have been acquired. These delays make the terminal output easier to observe and introduce variability into the interleaving of threads.

Random delays are for demonstration rather than correctness. The algorithm must remain safe even if delays are removed, made equal, or happen to align. The random-number generator should also be seeded once in the main thread if different delay sequences are desired between program runs. Calling `srand()` independently from every thread is not recommended without

careful handling, because threads may seed from the same time value and produce correlated sequences.

5.6 Thread Creation and Joining

The main thread creates each philosopher using `pthread_create`. Each call receives a thread handle, optional thread attributes, the function to execute, and a pointer to that philosopher's identifier. The identifier storage must remain valid until the worker has read it; using a stable array avoids the common mistake of passing the address of a loop variable that changes during thread creation.

After creation, the main thread calls `pthread_join` for each worker. Joining prevents the main thread from destroying shared mutexes or exiting before workers finish. In robust code, the return value of thread creation should be checked, and only successfully created threads should be joined.

5.7 Complexity Analysis

Let N be the number of philosophers and K the number of cycles per philosopher. Initialising the mutex array and creating the threads require $O(N)$ operations. Each philosopher performs a constant number of lock and unlock operations per cycle, so the total number of synchronization operations is $O(NK)$. The core program stores N thread handles, N mutexes, and N identifiers, giving $O(N)$ space complexity.

This asymptotic analysis describes the amount of work initiated by the program. Actual completion time also depends on the simulated delays, lock contention, thread scheduling, and the number of CPU cores. Because the delays dominate the useful work in this demonstration, runtime should not be interpreted as a benchmark of mutex performance.

6. Implementation

6.1 Implementation Overview

The implementation is written in C and uses the pthread API. A philosopher's identifier determines the fork indices and the acquisition order. The worker function is shared by all philosopher threads, while the main function handles initialisation, creation, joining, and cleanup.

The listing below presents the core worker logic used in the project. It is intended to show the central algorithm; the complete source file should include the declarations for the shared fork array and philosopher count, as well as a main function that initialises and destroys the mutexes.

```
1  void* philosopher(void* num) {
2      int id = *(int*)num;
3      int left_fork = id;
4      int right_fork = (id + 1) % NUM_PHILOSOPHERS;
5
6      for (int i = 0; i < EAT_CYCLES; i++) {
7          printf("Philosopher %d is THINKING.\n", id);
8          sleep(rand() % 2 + 1);
9
10         /* Deadlock prevention: asymmetric acquisition */
```

```
11          if (id % 2 == 0) {
12              pthread_mutex_lock(&forks[right_fork]);
13              pthread_mutex_lock(&forks[left_fork]);
14          } else {
15              pthread_mutex_lock(&forks[left_fork]);
16              pthread_mutex_lock(&forks[right_fork]);
17          }
18
19          /* Critical section: both forks are held */
20          printf("Philosopher %d is EATING.\n", id);
21          sleep(rand() % 2 + 1);
22
23          /* Release both resources */
24          pthread_mutex_unlock(&forks[left_fork]);
25          pthread_mutex_unlock(&forks[right_fork]);
26
27          printf("Philosopher %d finished eating and put down forks.\n", id);
28      }
29
30      printf("Philosopher %d has left the table.\n", id);
31      return NULL;
32  }
```

Listing 4: Philosopher thread function using asymmetric locking

6.2 Explanation of the Worker Function

The first statement converts the generic pointer argument into a philosopher identifier. The left fork is assigned the same index as the philosopher, while the right fork is the next index with wraparound. This mapping ensures that neighbouring philosophers share exactly one fork.

The loop runs for `EAT_CYCLES` iterations. At the start of each iteration, the philosopher announces that it is thinking and sleeps for a short random period. It then locks both forks. The parity check determines which fork is acquired first. The eating message is printed only after both lock calls have returned, so the message represents entry into the eating section.

After the eating delay, the thread unlocks both mutexes and reports that it has put the forks down. Once all iterations are complete, it prints a completion message and returns. The function returns a null pointer because no result value needs to be communicated to the main thread.

6.3 Important Implementation Considerations

6.3.1 Shared Output

Multiple threads can print to the same terminal. Standard I/O functions generally provide internal stream locking, but the sequence of complete messages can still vary, and messages from different threads may appear in a different order from run to run. If the output must be machine-readable, a dedicated output mutex or structured logging mechanism can be added.

6.3.2 Random Number Generation

The C library's traditional `rand()` function uses shared generator state and is not an ideal choice for independent random streams in multithreaded code. For a small educational simulation it may be acceptable, but a more rigorous implementation should use a thread-safe random source or provide separate per-thread generators. The random delay is not part of the deadlock-prevention proof.

6.3.3 Error Handling

A robust implementation should check the result of every `pthread_mutex_init`, `pthread_create`, `pthread_join`, lock, unlock, and destroy call. If initialising a mutex fails partway through, only the mutexes already initialised should be destroyed. If thread creation fails, the main thread must join the threads that were successfully created before cleaning up shared resources.

6.3.4 Correct Identifier Passing

When starting several threads in a loop, the program should pass a pointer to a distinct identifier element for each thread. Passing the address of one loop variable to every thread may cause several threads to read the same or an unexpected value. A fixed array such as `philosopher_ids[]` avoids that issue.

6.4 Compilation and Execution

The source can be compiled using GCC on Linux:

```
gcc dining_philosophers.c -o dining -pthread
./dining
```

Listing 5: Build and run commands

If the compiler reports that a pthread symbol is undefined, the build command should be checked to ensure that `-pthread` is included. If the compiler reports an undeclared function such as `sleep`, the source should include the appropriate header, commonly `unistd.h`. The program should also include `pthread.h`, `stdio.h`, `stdlib.h`, and any other headers used by the full source.

6.5 Suggested Complete-Program Structure

A complete source file typically contains the following elements in order:

1. Header files for standard I/O, random delays, and pthreads.
2. Constants defining the number of philosophers and cycles.
3. Global or shared declarations for the fork mutexes.
4. The philosopher thread function.
5. A `main()` function that initialises mutexes and identifiers.
6. A loop that creates philosopher threads.

7. A loop that joins the successfully created threads.

8. A cleanup loop that destroys mutexes and a final completion message.

Keeping initialisation and cleanup in `main()` makes ownership of shared resources easier to understand. The worker function should focus on the philosopher lifecycle and should return normally after completing its cycles.

7. Testing and Results

7.1 Testing Objectives

Testing checks whether the simulator satisfies the functional requirements and whether its output is consistent with the intended synchronization design. Since thread schedules are nondeterministic, one successful run is not enough to prove every possible interleaving correct. The program should be run repeatedly and reviewed against explicit properties.

The key properties are:

- No philosopher prints the eating message before acquiring both forks.
- Each philosopher completes the configured number of cycles under normal execution.
- No fork is simultaneously used by both philosophers that share it.
- The program terminates rather than remaining permanently blocked.
- Mutexes are destroyed only after the worker threads have finished.

7.2 Build Test

The build test verifies that the source compiles with pthread support and that the executable starts. A representative terminal session is shown below. Replace the sample output with the actual output captured from the final version of the program before submission.

```
$ gcc dining_philosophers.c -o dining -pthread
$ ./dining
Starting Dining Philosophers Simulation...
```

Listing 6: Compilation and launch

7.3 Sample Simulation Output

The exact ordering of messages varies because the threads execute concurrently. The following is an illustrative excerpt, not a guaranteed output transcript.

```
Starting Dining Philosophers Simulation...

Philosopher 0 is THINKING.
Philosopher 1 is THINKING.
Philosopher 2 is THINKING.
Philosopher 3 is THINKING.
Philosopher 4 is THINKING.
```

```
Philosopher 0 is EATING.
Philosopher 0 finished eating and put down forks.
Philosopher 2 is EATING.
Philosopher 1 is EATING.
Philosopher 0 is THINKING.
Philosopher 2 finished eating and put down forks.
Philosopher 3 is EATING.
Philosopher 1 finished eating and put down forks.
...
Philosopher 4 has left the table.

Simulation complete. All philosophers ate successfully.
```

Listing 7: Illustrative terminal output

The excerpt illustrates the intended message format and state transitions. The ellipsis represents omitted lines and must not be presented as a full real execution log. For a final academic report, a screenshot or a complete text capture from an actual run can be placed here.

7.4 Test Cases

Test	**Action**	**Expected result**	**Status**
T-01	Compile using GCC and pthread support.	Compilation succeeds without errors.	To be verified
T-02	Start the program with five philosophers.	Five worker threads are created under normal conditions.	To be verified
T-03	Observe fork acquisition and eating messages.	Eating is announced only after both forks have been acquired.	To be verified
T-04	Run the simulation to completion.	Each philosopher completes the configured number of cycles.	To be verified
T-05	Repeat the program several times.	Message ordering may vary, but the program should complete without deadlock.	To be verified
T-06	Review resource cleanup.	The main thread joins workers before destroying mutexes.	To be verified
T-07	Inspect fairness over multiple runs.	No starvation claim is made solely from random delays; observed waiting can be recorded.	To be verified

Table 3: Proposed functional and behavioural test cases.

7.5 Results

The implementation demonstrates concurrent execution through interleaved philosopher messages. Mutex locks ensure that a fork is not simultaneously acquired by two threads. The asymmetric acquisition policy breaks the circular-wait condition in the designed arrangement, so the system is expected to complete without the classic deadlock in which every philosopher holds one fork and waits for another.

The program's finite cycle count makes successful termination observable. Once each philosopher finishes its loop, the main thread's join operations return and cleanup can proceed. The final output should be checked against the actual run before reporting that all philosophers completed successfully.

7.6 Interpretation of Results

The most important result is the separation of responsibilities. Mutexes enforce exclusive ownership of individual forks, while the acquisition protocol prevents the circular dependency pattern. Neither component alone is a complete explanation of correctness: mutexes can prevent simultaneous use of a fork but still permit deadlock if threads acquire several locks in a problematic order.

The varying order of terminal messages is normal. It reflects scheduling and the relative timing of delays rather than an error. A thread that prints later is not necessarily less correct, and the order of console lines should not be treated as a fixed expected sequence.

7.7 Limitations of the Evaluation

The simulation is small and uses artificial delays. It does not collect precise waiting-time statistics, measure throughput, or prove starvation freedom. Terminal messages provide qualitative evidence, but they are not a substitute for formal verification or instrumentation. More detailed evaluation could record the time each philosopher spends waiting for forks, the number of successful meals, and the maximum observed wait across many runs.

8. Discussion

8.1 Mutual Exclusion in Practice

The project illustrates why a shared resource needs an explicit ownership rule. A fork is represented by one mutex, so a second thread cannot successfully lock that mutex until it is released. Since the eating section begins only after both locks have been acquired, two adjacent philosophers cannot eat simultaneously while using their shared fork.

This does not mean that only one philosopher can eat at a time. Philosophers who do not share forks may eat concurrently. For example, philosopher 0 and philosopher 2 do not share a fork in the five-philosopher arrangement and may be able to eat at the same time, depending on the schedule and resource availability. This concurrency is one reason the problem is more interesting than placing a single global lock around the entire eating operation.

8.2 Deadlock Prevention Versus Deadlock Detection

Deadlock prevention changes the rules so that at least one necessary deadlock condition cannot occur. In this project, the ordering rule is intended to prevent circular wait. Deadlock detection, by contrast, allows a deadlock to occur and then attempts to identify it, for example by analysing resource-allocation state. Recovery may require terminating a thread or releasing resources.

Prevention is appropriate for this small simulation because the policy is simple and has low overhead. In more complicated applications, a combination of lock ordering, timeouts, detection,

and recovery may be necessary. The correct choice depends on the resource model and the cost of blocking or restarting operations.

8.3 Starvation and the Limits of Random Delays

The original report objective included starvation prevention. It is important to state the guarantee precisely. Randomised thinking and eating intervals can reduce repeated synchronisation patterns and make it less likely that all threads behave in lockstep. However, the scheduler and mutex implementation are not required to grant each philosopher access in strict first-in-first-out order. Randomness cannot prove that every philosopher will be selected within a bounded time.

A starvation-resistant design could use a central waiter that records requests and grants permission only when both forks are available. The waiter could maintain a FIFO queue to reduce the chance that a philosopher is repeatedly bypassed. Another design could use a monitor with condition variables and explicit state transitions. These approaches add complexity but allow fairness to be addressed directly.

8.4 Scheduling and Nondeterminism

The operating system scheduler decides which runnable thread receives CPU time. The program does not control the exact order in which philosophers wake up, attempt mutex acquisition, or print messages. A different execution may produce a different sequence even on the same machine. This nondeterminism is a defining characteristic of concurrent programming.

The correctness criteria should therefore be based on invariants rather than a particular output order. It is reasonable to expect each philosopher to complete its finite loop in normal runs, but the report should avoid claiming that a single observed transcript proves correctness for all schedules.

8.5 Performance Considerations

For five philosophers, performance is not a practical concern. Mutex operations have overhead, and threads may spend much of their time sleeping or blocked on locks. The program is a teaching tool rather than a benchmark. Increasing the number of philosophers could reveal additional contention and make the output harder to interpret.

If the simulation were extended to collect performance metrics, useful measurements would include total elapsed time, average waiting time, maximum waiting time, meals per philosopher, and the number of times a philosopher blocks while acquiring a fork. Such metrics would help compare asymmetric ordering with a waiter or monitor solution.

8.6 Reliability and Defensive Programming

The core algorithm is short, but real concurrent programs must consider error paths. A mutex initialisation error should not be ignored. A failed thread creation means that fewer workers exist than expected, and cleanup must join only those that were actually created. If a thread acquires one fork and then encounters an error before acquiring the second, it must release the first fork before returning.

A more robust implementation can use a cleanup path or carefully structured acquisition logic. It should also ensure that the same mutex is never locked twice by the same thread and

that every successful lock is eventually matched by an unlock. These practices are important even when the happy path works correctly.

9. Advantages, Limitations, and Applications

9.1 Advantages

- **Clear demonstration:** The problem provides a visual and conceptual way to understand concurrency.
- **Practical use of pthreads:** The program demonstrates thread creation, joining, and mutex operations.
- **Low implementation overhead:** The asymmetric rule requires only a small amount of additional logic.
- **Deadlock prevention:** The policy breaks circular wait for the specified arrangement.
- **Observable behaviour:** Console messages show concurrent state changes.
- **Extensibility:** The same foundation can be extended with statistics, graphical output, or alternative algorithms.

9.2 Limitations

- The number of philosophers and cycles is fixed at compile time.
- Random sleep intervals are not a formal starvation-prevention mechanism.
- The sample implementation may not check every pthread return value.
- Console output does not provide exact waiting-time or throughput measurements.
- The program does not include a graphical user interface.
- A finite demonstration cannot by itself prove correctness for every possible thread schedule.

9.3 Real-World Applications

The concepts illustrated by the simulator apply to a wide range of systems:

- **Database transactions:** Transactions may need locks on several rows or tables. Consistent lock ordering can reduce deadlock risk.
- **Operating-system resource management:** Processes may compete for devices, memory regions, or kernel objects.
- **File processing:** Threads may need exclusive access to multiple files or shared metadata.
- **Network services:** A request may depend on several shared buffers, connection objects, or queues.

- **Concurrent data structures:** Operations that acquire more than one lock must consider lock ordering and possible cycles.
- **Embedded systems:** Tasks may share sensors, buses, and peripherals that require exclusive access.

9.4 Learning Outcomes

By completing the project, students gain practical experience with the relationship between threads and shared resources. They learn to identify critical sections, use mutexes, reason about the Coffman conditions, distinguish deadlock from starvation, and interpret nondeterministic output. The project also highlights the importance of careful cleanup and error handling in multithreaded programs.

10. Future Enhancements

10.1 Configurable Number of Philosophers

The program could accept the number of philosophers and the number of cycles as command-line arguments. Dynamic allocation would allow the arrays of mutexes and thread handles to scale with the selected input. Input validation would be required to reject invalid values such as zero philosophers or negative cycle counts.

10.2 Graphical Visualisation

A graphical interface could show the table, philosophers, forks, and current states. Each philosopher could change colour when thinking, waiting, eating, or finished. A visual interface would make resource ownership easier to understand and could include a pause button and step-by-step execution mode.

10.3 Waiting-Time Statistics

The program could use a monotonic clock to record when each philosopher begins waiting and when it acquires the second fork. The difference would provide a waiting-time measurement. Over multiple runs, the program could report average and maximum wait, number of meals, and total execution time. Care must be taken to protect shared statistics with a mutex or to store per-thread results separately.

10.4 Fair Waiter Algorithm

A waiter-based design could accept requests and grant permission only when the necessary resources are available. If requests are queued in first-in-first-out order, the implementation can make a clearer fairness argument than one based on random delays. The design must still be analysed for correctness and should avoid holding a central lock while a philosopher performs the entire eating delay.

10.5 Alternative Synchronization Strategies

The simulator could include selectable modes: naive locking, asymmetric locking, global resource ordering, semaphore admission control, and a monitor solution. A comparison mode would help demonstrate how different strategies affect deadlock, concurrency, waiting time, and code complexity.

10.6 Improved Error Handling and Instrumentation

Future versions should check pthread return values, recover from partial initialisation, and ensure all acquired resources are released on every exit path. Instrumentation could record lock acquisition and release events in a structured log, making it possible to replay or analyse execution traces.

11. Conclusion

This project implements a terminal-based Dining Philosophers simulator in C using POSIX threads and mutex locks. The five philosopher threads share five fork mutexes and repeatedly alternate between thinking and eating. A philosopher enters the eating section only after acquiring both adjacent forks, thereby enforcing the resource ownership rule.

The asymmetric acquisition policy is the central deadlock-prevention mechanism. Even-numbered philosophers acquire the right fork first, while odd-numbered philosophers acquire the left fork first. By breaking the uniform circular ordering of resource requests, the policy prevents the classic circular-wait deadlock in the defined arrangement. The main thread joins the worker threads before destroying the mutexes, ensuring that shared synchronization objects remain available for the duration of worker execution.

The project also demonstrates an important limitation: randomized delays make execution more varied, but they do not formally guarantee starvation freedom. Deadlock prevention and starvation prevention are separate goals and require separate reasoning. A future version could add a FIFO waiter, a monitor, waiting-time measurements, and a graphical interface to make fairness and resource allocation easier to study.

Overall, the simulator provides a practical demonstration of concurrency, mutual exclusion, deadlock prevention, thread lifecycle management, and the importance of designing synchronization protocols rather than relying on locks alone.

Bibliography

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: John Wiley & Sons, 2018.

[2] E. W. Dijkstra, "Cooperating sequential processes," Technological University Eindhoven, Eindhoven, The Netherlands, Rep. EWD123, 1965.

[3] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA: No Starch Press, 2010.

[4] D. R. Butenhof, *Programming with POSIX Threads*. Reading, MA: Addison-Wesley, 1997.

[5] IEEE and The Open Group, *The Open Group Base Specifications, Issue 7: POSIX.1-2017*, 2018. Available: https://pubs.opengroup.org/onlinepubs/9699919799/.

[6] Linux man-pages project, "pthreads(7), pthread_create(3), pthread_join(3), and pthread_mutex_lock(3)," Linux manual pages. Available: https://man7.org/linux/man-pages/.